Davante

15

PROGRAMACIÓN

Depuración y pruebas de programa

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Ciclo de vida del software 5

2.1. Fases del desarrollo 5

/ 3. Depuración del software 5

3.1. Métodos de depuración del software 6

3.2. Proceso de depuración del software 6

3.3. Herramientas de depuración software 7

/ 4. Consideraciones sobre la depuración del software 7

4.1. Estrategias de depuración del software 7

4.2. Ventajas de la depuración del software 8

4.3. Desventajas de la depuración software 8

/ 5. Calidad del software 9

/ 6. Caso práctico 1: "Error indetectable" 9

/ 7. Depuración de programas 10

7.1. El depurador de Netbeans 11

7.2. Puntos de ruptura 12

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 8. Caso práctico 2: “Sesión de pruebas” 13

/ 9. Pruebas del software 13

9.1. Pruebas de caja blanca 14

9.2. Pruebas de caja negra 14

9.3. Desarrollo guiado por pruebas (TDD) 14

/ 10. Resumen y resolución del caso práctico de la unidad 15

/ 11. Bibliografía 16

OBJETIVOS

Conocer los tipos de pruebas para programas.

Conocer el concepto de depuración.

Saber utilizar el depurador.

Conocer los principios de la calidad del software.

/ 1. Introducción y contextualización práctica

El desarrollo de *software* se centra en generar un producto que pueda ser utilizado por los usuarios finales con seguridad. Para ello, es necesario que cuando se genere un producto software se establezcan ciertas medidas que permitan garantizar que no existen errores en su implementación. Como programadores podemos realizar ciertas tareas que permitan mejorar la calidad del producto y mejore la experiencia de usuario.

Las pruebas y depuración del software forman parte del ciclo de vida del producto. Existen metodologías que centran todo el desarrollo en las pruebas, es decir, están guiadas por el proceso de prueba. Cuando trabajamos con pruebas, es necesario que realicemos determinados cambios en el producto que permitan probar con facilidad. Es responsabilidad del programador generar las versiones del programa que hagan posibles las pruebas correspondientes.



Fig.1. Depuración de programas.

En este tema se presentan los principales aspectos que debemos tener en cuenta para generar un producto viable y de calidad para el usuario final.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Elección de tipo de pruebas"

https://on.soundcloud.com/8aZgokC89MGN9pXZA

/ 2. Ciclo de vida del software

El desarrollo de componentes software se centra en crear componentes/programas que puedan ser utilizados por los usuarios. Para poder llevar a cabo un producto correctamente, es necesario que atendamos a las diferentes fases por las que puede pasar el desarrollo software. Normalmente, hablamos de ciclo de vida del software para referirnos a todas aquellas **fases por las que pasa el desarrollo**. Podemos decir que el ciclo de vida del software tiene en cuenta todas las fases necesarias para poder lograr un desarrollo válido del mismo. En teoría, si realizamos todas las fases del ciclo de vida obtendremos un producto que satisfaga todos los requisitos del cliente y cumpla con todos los procedimientos de desarrollo.

El ciclo de vida del software está dividido en diferentes fases que determinan, con suficiente claridad, qué tareas llevar a cabo en cada una de ellas. Es necesario destacar que las etapas pueden verse alteradas por la utilización de una metodología de desarrollo frente a otra.

2.1. Fases del desarrollo

La creación del software se puede ver como un modelo sistemático en el que la forma de desarrollar el proyecto está acotada. Normalmente, existen una serie de **fases comunes** por las que pasan todos los proyectos software.

Cuando implementamos una metodología en el desarrollo de un proyecto, estamos acotando y definiendo las fases y tareas que se realizan en cada una de ellas. Por este motivo, es sumamente importante que podamos **definir cuanto antes la metodología de desarrollo**.

Las **fases** del desarrollo software son:

- **Planificación**: Se establecen las fechas e hitos a alcanzar en cada una.
- **Análisis**: Permite determinar qué tareas es necesario realizar.
- **Diseño**: Establece cómo efectuar las tareas para ser más eficientes.
- **Implementación**: Lleva a cabo las tareas necesarias.
- **Pruebas**: Se centra en comprobar las tareas realizadas.
- **Despliegue**: Se centra en instalar el sistema que hayamos creado.



Fig.2. Ciclo de vida del software.

- **Mantenimiento**: Permite establecer un mecanismo de corrección de errores y ampliación del sistema.

/ 3. Depuración del software

Cuando creamos un sistema software, en ocasiones, podemos encontrar problemas, durante la implementación, que necesitan de una visión detallada del flujo de ejecución para observar con precisión lo que ocurre durante el proceso de ejecución. En este contexto cobra especial interés introducir el proceso de depuración.

La depuración permite identificar y resolver errores o fallos en el código con cierta facilidad. Se trata de una herramienta sumamente importante en el desarrollo software. Si bien es cierto que, a medida que aumenta la experiencia de los desarrolladores, se vuelve menos necesaria y es utilizada solo en ciertas ocasiones. Gracias a la depuración, los desarrolladores pueden asegurar que los componentes funcionan correctamente. Aunque es una gran herramienta para los desarrolladores, suele requerir un tiempo considerable para poder ejecutarse y obtener resultados que ayuden a solucionar los problemas necesarios.

A continuación una curiosidad sobre el proceso de depuración:

> **¡! Sabías que...**
>
> El proceso de depuración suele ayudar mucho a la localización de errores y posible resolución de los estos. La mayor parte de los IDEs ofrecen herramientas integradas para poder depurar el código fuente.

3.1. Métodos de depuración del software

El proceso de depuración del software se puede realizar utilizando diferentes técnicas y métodos. A continuación, se muestran algunas de las alternativas más utilizadas en la actualidad:

- **Inspección del código**: Este método requiere de la revisión manual de los desarrolladores para intentar comprender dónde puede estar el error.
- **Trazas**: Consiste en añadir líneas de código al programa que permitan dejar registros de cómo se comporta el flujo de ejecución.
- **Herramientas de depuración**: Se trata de un conjunto de herramientas destinadas a realizar la depuración software del código.
- **Test unitarios**: Es una técnica que permite probar los diferentes caminos de ejecución por los que pasa el software.
- **Test de integración**: Permite probar cómo se comporta el software cuando se relaciona con otros componentes del sistema.



Fig.3. Un fallo en el software se conoce como 'bug'.

- **Monitorización**: Hace posible establecer un mecanismo de observación del software durante su ejecución.

3.2. Proceso de depuración del software

Para depurar el software es necesario seguir una serie de pasos que nos permitirán realizar el proceso **teniendo en cuenta la estrategia** que hayamos elegido.

En el proceso de depuración del software podemos seguir los siguientes pasos:

- **Identificar el problema a solucionar**: Es necesario determinar el tipo de informe que queremos generar.
- **Asignar el proceso de depuración** a uno de los miembros del equipo.
- **Analizar los errores generados** para poder encontrar formas de reproducir el error fácilmente: Mediante la reproducción del error podremos determinar sus causas, contextos, etc. Es fundamental ser capaces de reproducir los errores para poder solucionarlos fácilmente. Los que se producen de forma aleatoria (sin seguir un patrón determinado) pueden ser muy complicados de depurar.
- **Una vez encontrado el error, realizar la implementación que lo corrige**.
- **Comprobar que la corrección soluciona el error** sin introducir nuevos errores.

3.3. Herramientas de depuración software

Cuando hablamos de herramientas de depuración nos centramos en aquella estrategia que nos permite establecer un proceso de depuración. La herramienta fundamental para llegar a cabo la depuración es el **depurador de código**.

El depurador **es una herramienta software que permite a los desarrolladores observar el flujo de ejecución con un gran nivel de detalle**. Dependiendo del contexto en el que estemos desarrollando, puede ser interesante un tipo de datos frente a otros. Por ejemplo, cuando estamos trabajando con dispositivos que disponen de una batería, puede interesarnos observar cómo las diferentes instrucciones de código pueden afectar a la duración de la batería del dispositivo. En el caso del desarrollo web, puede ser interesante observar cómo se cargan los elementos en el navegador y determinar si se van a ver exactamente igual en todos los navegadores. Normalmente, los IDE ofrecen diferentes herramientas para poder llevar a cabo el proceso de depuración.

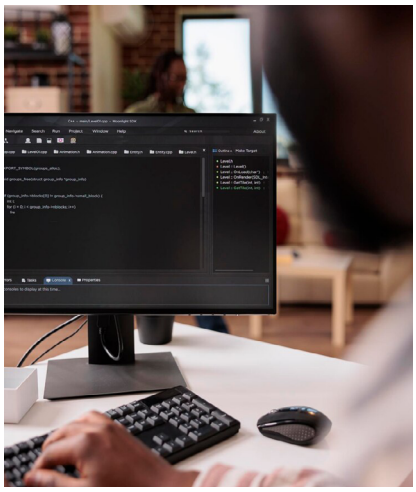


Fig.4. Depuración de programas.

Audio 1. "Uso del depurador"
https://on.soundcloud.com/XnP8b8v5d3b6ciZP7

/ 4. Consideraciones sobre la depuración del software

Cuando nos enfrentamos a la depuración de un programa, es importante tener en cuenta algunos aspectos que nos pueden ayudar a la hora de gestionar el proceso de depuración. Por ejemplo, tenemos que ser capaces de elegir la estrategia de depuración que más se adapte a nuestro contexto o al tipo de problema que estamos intentando solucionar.

Recuerda...

Es necesario adaptar las estrategias de depuración y pruebas a los recursos disponibles en nuestro equipo. Además, es fundamental recordar que cada estrategia de depuración puede tener un coste asociado.

4.1. Estrategias de depuración del software

En el proceso de depuración del software podemos encontrarnos con diferentes estrategias que se adaptan mejor a cada tipo de situación. A continuación, se describen algunas de las más utilizadas:

- **Fuerza bruta**: Se trata de un tipo de depuración que requiere un gran esfuerzo, porque hace necesaria una comprensión total del sistema. Normalmente, utilizamos este tipo de estrategias **cuando no tenemos claro el problema que se está produciendo**.

- **Eliminación**: Esta estrategia de depuración requiere que se conozca el código y los datos de entrada que se pueden utilizar. El objetivo es minimizar el conjunto de datos de entrada y posibles partes del código que puedan generar el problema utilizando un **proceso de partición**.

- **Análisis hacia delante**: Permite establecer trazas en el código mediante puntos de parada en el depurador. Nótese que un depurador permite parar el flujo de ejecución del código allí donde sea necesario y continuar cuando el programador lo estime oportuno. Mediante esta estrategia podemos **parar el código y analizar lo que está sucediendo** en determinados módulos.

- **Análisis hacia atrás**: Hace posible establecer puntos de parada hacia atrás. **Es necesario que el depurador nos permita volver atrás en el flujo de ejecución**. En ocasiones no es posible aplicar esta estrategia porque el depurador no permite esta funcionalidad.



Fig.5. Pruebas por fuerza bruta.

- **Depuración automática**: Se centra en utilizar **herramientas que faciliten la depuración de manera asistida**. Gracias a este tipo de herramientas es posible emplear estrategias de inteligencia artificial para encontrar posibles causas de errores más fácilmente.

4.2. Ventajas de la depuración del software

Como hemos visto, el depurador ofrece una serie de ventajas durante el desarrollo del software. A continuación, podemos enumerar alguna de las principales **ventajas que nos encontramos durante la utilización del depurador**:

- **Mejora de la calidad del sistema**: Gracias al uso del depurador es posible solventar errores que pueden afectar a la eficacia y estabilidad de nuestro sistema.

- **Disminución del tiempo de espera**: En ocasiones, algunos de los errores que se pueden producir tienen que ver con el tiempo requerido para llevar a cabo una determinada acción.

- **Solución de errores**: Gracias al depurador podemos solventar este tipo de errores y ofrecer un sistema más estable y con menor tiempo de respuesta.

- **Mejora de la satisfacción del usuario**: La utilización del depurador hace posible mejorar la satisfacción del usuario ofreciendo un sistema que es más fácil de usar y que satisface todos los requisitos necesarios por parte del usuario.

- **Reducción de costes**: Al emplear un depurador podemos disminuir el coste de desarrollo, porque es posible detectar los errores con mayor facilidad.

- **Mejora de la seguridad**: El uso del depurador puede identificar problemas que pueden poner en riesgo la seguridad del sistema. Por ejemplo, facilita detectar posibles fugas de datos que expondrían los datos de los usuarios a terceras partes.

4.3. Desventajas de la depuración software

Aunque el depurador es una herramienta muy potente para los desarrolladores, en ocasiones se presentan algunas desventajas.

Por ejemplo, cuando utilizamos el depurador, debemos tener en cuenta que es una tarea que requiere un **alto tiempo**. Si invertimos mucho tiempo en la depuración, podrían **aumentar los costes** de nuestro proyecto. Otro problema que podemos destacar a la hora de utilizar el depurador es la necesidad de un **personal especializado** para poder interpretar los datos necesarios que se ofrecen en el proceso de depuración.



Fig.6. Ventajas y desventajas de la depuración del software.

/ 5. Calidad del software

Cuando nos enfrentamos a un producto que tenemos que proporcionar a nuestros clientes, debemos ser capaces de garantizar unos **estándares de calidad**. Desde el punto de vista del software, la calidad es una de las tareas a las que se dedican muchos esfuerzos. A pesar de ello, es importante tener en cuenta que es prácticamente imposible lograr un producto software que esté libre de errores. Las diferentes metodologías de desarrollo y pruebas nos pueden ayudar a minimizar el número de errores que tengamos en el producto.

La calidad del software va a depender de la calidad de cada una de las tareas que implica su construcción. Por ejemplo, hablamos de calidad de los requisitos, calidad del análisis, calidad del diseño, etc. No hay que confundir la calidad del software (producto) con la calidad de la implementación, pues se trata de una fase más del proceso de implementación del software.

Otra dimensión de la calidad del software tiene que ver con el **mantenimiento**. Por ello, es sumamente importante que podamos desarrollar software que sea fácilmente mantenible y que pueda evolucionar fácilmente.

Recuerda...

La calidad del software es fundamental para tener un producto viable en la realidad. Es importante satisfacer las necesidades de nuestros clientes para poder disponer de un producto sostenible en producción.

Para poder establecer unos estándares de calidad en el software es necesario que se siga una gestión de la calidad adecuada a la naturaleza del producto que estamos generando. Normalmente, podemos identificar **tres etapas para gestionar correctamente la calidad del software**:

- **Planificación**: Permite registrar los requisitos y las diferentes normas de calidad que deben cumplirse en el proyecto que llevamos a cabo. Utilizará la documentación para demostrar que se ha satisfecho el nivel de calidad requerido. Esta etapa puede requerir la identificación de requisitos, la definición de métricas/estándares, etc.
- **Gestión**: Se centra en hacer cumplir las diferentes normas y objetivos en el plan general de calidad.
- **Control**: Permite supervisar y mantener un registro de las diferentes actividades relacionadas con la gestión que se llevan a cabo. En esta fase se pueden destacar aspectos como el desarrollo basado en pruebas, la definición de estrategias de comunicación, etc.



Fig.7. Calidad del software.

/ 6. Caso práctico 1: “Error indetectable”

Planteamiento: Tras sacar un programa a producción, observamos que todo funciona correctamente. Al pasar un mes de su puesta en marcha, comienzan a surgir errores continuos, pues el sistema sufre cortes constantes en el funcionamiento. En ese momento, decidimos comenzar a depurar el programa. Sin embargo, no somos capaces de reproducir el problema que está surgiendo en producción. Nuestro jefe nos pide que cojamos la versión de producción y depuremos sobre ella.

Nudo: ¿Crees que tiene sentido lo que se nos plantea? ¿Cómo lo solucionarías?

Desenlace: Cuando nos disponemos a sacar un producto a producción debemos considerar este tipo de situaciones. Se pueden dar situaciones reales en las que el software comienza a fallar y no somos capaces de reproducir el problema cuando comenzamos a realizar pruebas. Es importante tener en cuenta que una versión de producción, si está bien creada, no puede depurarse, pues no contiene los símbolos necesarios en el binario de ejecución. Sería necesario crear una versión que permita comenzar el proceso de depuración. Por ello, lo que nos comenta nuestro jefe no tiene sentido.

Para intentar solucionar este tipo de situaciones, podemos añadir trazas al producto que puedan activarse/desactivarse a demanda, según sean las necesidades. En la actualidad existen servicios de grandes compañías que permiten realizar esta tarea de una forma muy sencilla y de manera remota. Es una buena práctica en el desarrollo software que consideremos la posibilidad de añadir trazas sobre el flujo de ejecución del programa. De esta forma, si sucede algo en ejecución, podemos analizar por dónde pasa la ejecución e intentar dar una solución al problema.

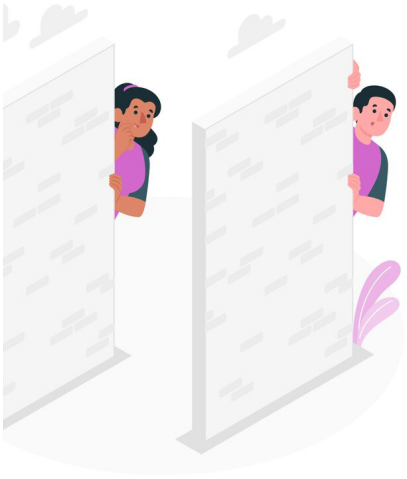


Fig.8. Error indetectable.

/ 7. Depuración de programas

En los programas vamos a encontrarnos con **errores de código**, ya sean **en tiempo de compilación**, en cuyo caso nos avisará el propio compilador, hasta errores **en tiempo de ejecución**, de los que el compilador no nos avisará. En cuanto a estos últimos, la única forma que vamos a tener de descubrir dónde están es haciendo que se provoquen.

Puede ser que estos errores no aparezcan en una primera ejecución, sino pasado un tiempo y al introducir unos datos específicos en nuestros programas, o puede ser que sí se presenten desde un inicio, pero que se produzcan unas veces sí y otras no, siendo muchísimo más complicada su detección, y, por ende, el poder solucionarlos. Para ayudarnos a lidiar con estos errores, aparte de la experiencia que podamos ir adquiriendo, tenemos a los **depuradores de código**.



Fig.9. Botón para lanzar el depurador de NetBeans.

Las **operaciones que nos va a permitir realizar un depurador** son las siguientes:

- **Establecimiento de puntos de interrupción:** Podemos establecer puntos de interrupción en el código para detener la ejecución del programa en un lugar específico y examinar el estado de las variables y la pila de llamadas en ese momento.
- **Ejecución paso a paso:** El depurador permite ejecutar el programa paso a paso, lo que hace posible observar cómo se ejecuta línea por línea y cómo cambian las variables en cada paso.
- **Inspección de variables:** Podemos examinar el valor de las variables en cualquier momento durante la ejecución del programa, lo que permite verificar si los datos se están procesando correctamente.
- **Evaluación de expresiones:** El depurador facilita evaluar expresiones en tiempo de ejecución, lo que permite verificar el resultado de cálculos o expresiones en diferentes puntos de tu código.
- **Seguimiento de la pila de llamadas:** Podemos ver la pila de llamadas actual para entender cómo se ha llegado a un determinado punto en el código y qué funciones se han llamado en el camino.

- **Examen y modificación del flujo de ejecución:** Al detener la ejecución del programa en un punto específico, podemos examinar y modificar el flujo de ejecución, como saltar líneas de código o avanzar hacia atrás en la ejecución.

- **Control de la ejecución:** Podemos iniciar, detener, pausar y reanudar la ejecución del programa según sea necesario durante la depuración.

- **Monitoreo de excepciones:** El depurador nos permite detectar y manejar excepciones en tiempo de ejecución, lo que nos ayuda a identificar y corregir errores en nuestro código.

7.1. El depurador de Netbeans

Como hemos adelantado anteriormente, el IDE NetBeans ya trae integrado su propio depurador, siendo este bastante **completo y fácil de usar**.

Para ejecutarlo debemos pulsar el botón de depurador que se presenta como el símbolo del 'play' con una línea en rojo. En la siguiente figura, es el botón del centro.



Fig.10. Botón para lanzar el depurador de NetBeans.

Esta opción se encuentra justo a la derecha del botón de compilar programas.

Una vez pulsemos el botón del depurador, comenzará el proceso y el programa se compilará y ejecutará, igual que cuando pulsamos el botón de compilar, solo que esta vez tendremos el depurador abierto.

Con este abierto, podremos controlar la línea de código que se va ejecutando, pasar a la siguiente línea, a la anterior, o al siguiente bloque que se vaya a ejecutar, habilitándose los siguientes botones:



Fig.11. Botones del depurador.

En orden de izquierda a derecha, tenemos:

- Botón para terminar el proceso de depuración.

- Botón para pausar el proceso de depuración.

- Botón para continuar el proceso de depuración.

- Botón para ir a la siguiente línea de código.

- Botón para ir a la siguiente función que se ejecutará.

También nos aparecerá la ventana de control de variables, donde visualizaremos todas las variables que están implicadas en la ejecución del código con sus respectivos valores. Podremos cambiar estos desde esta ventana si es necesario.

A continuación una figura sobre ventana de control de variables:

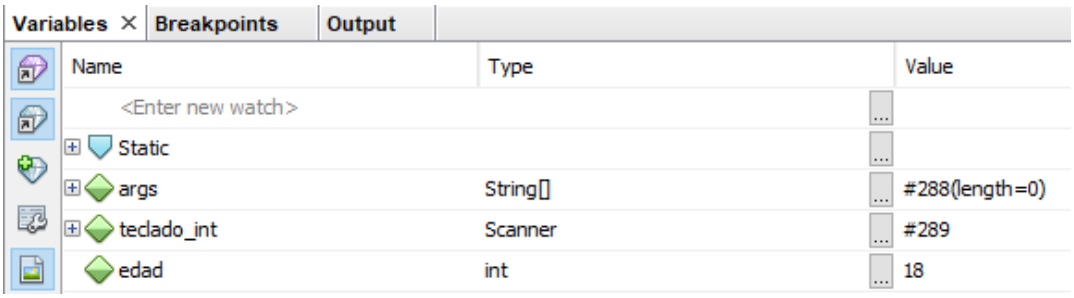


Fig.12. Ventana de control de variables.

7.2. Puntos de ruptura

Ya hemos visto que los puntos de ruptura (*breakpoint*) son una de las muchas herramientas que nos va a proporcionar cualquier depurador de código, entre ellos, el de NetBeans.

Indicando un punto de ruptura, vamos a conseguir que, cuando estemos depurando un programa, **el depurador se pare justo en esa línea**, de modo que podremos observar la ventana de control de variables para ver qué valores tienen, así podremos deducir posibles errores.

Podremos poner **tantos puntos de ruptura como líneas tenga nuestro programa**.

Una buena estrategia para la detección de errores es poner un punto de ruptura justo en la línea anterior a aquella donde se produce el error, así podremos observar los valores de las variables implicadas y será más fácil ver qué puede estar provocando el error.

Para colocar un punto de ruptura en un programa con NetBeans, debemos pulsar, en la parte izquierda, sobre el número de la línea donde lo queremos poner. Una vez hecho esto, la línea se pondrá de color rojo, indicando que hay un punto de ruptura. Para poder eliminar dicho punto, basta con volver a pulsar sobre el número de la línea y dejará de ser roja.

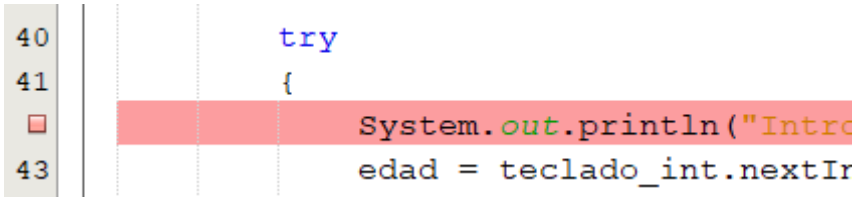


Fig.13. Punto de ruptura en la línea 42.

Cuando ejecutemos el depurador, se parará automáticamente en la línea 42, mostrando la ventana de control de variables y podremos ver qué variables hay implicadas y cuáles son sus valores en ese preciso momento.

Podemos poner puntos de ruptura en diferentes métodos de clases de nuestro programa, de modo que, cuando el depurador vaya llegando a ellos, se irá parando automáticamente.

Si ejecutamos normalmente un programa con puntos de ruptura, no ocurrirá nada especial, sino que el programa se ejecutará como si nada.

Vídeo 1. "Ejemplo de depuración de un programa"

https://bit.ly/2GtOkVg

/ 8. Caso práctico 2: "Sesión de pruebas"

Planteamiento: En el desarrollo de un componente software crítico para un sistema de vuelo, observamos que hay situaciones en las que el sistema de control de vuelo no reacciona correctamente. Nos percatamos de que existe una discordancia entre los datos de la velocidad del aire y la velocidad real. Como resultado, el sistema eleva el morro del avión, generando una entrada en pérdida (pérdida súbita de sustentación). Dado que se trata de un error grave, nuestro equipo decide montar una sesión de pruebas de forma urgente. Nuestro jefe nos pide que comencemos cuanto antes una sesión de depuración.



Fig.14. Test de carácter crítico.

Nudo: ¿Cómo enfocarías este problema? ¿Tiene sentido la propuesta de nuestro responsable?

Desenlace: Como podemos deducir de este caso, la secuencia de errores que se produce en este escenario es realmente grave, pues existen vidas humanas en riesgo. En este contexto, es fundamental que llevemos a cabo una sesión de pruebas exhaustiva, de tal forma que localicemos y solucionemos la fuente del error. Para ello, podemos establecer diferentes estrategias de prueba. Un equipo puede realizar pruebas de caja blanca y otro equipo pruebas de caja negra. Además del tipo de prueba, podemos adoptar diferentes clases de estrategias durante la depuración. Por ejemplo, podemos utilizar estrategias de fuerza bruta y también de eliminación.

Cuando nos enfrentamos a situaciones de errores críticos, debemos ser capaces de establecer una metodología de pruebas que permita localizar el error cuanto antes. Nótese que la localización del error suele ser más costosa que su resolución, por ello, es fundamental poner todos los recursos necesarios en la fase de localización.

/ 9. Pruebas del software

Son fundamentales para llevar a cabo la comprobación de los requisitos y funcionalidades que hemos implementado en nuestros programas. Podemos entender que las pruebas **permiten validar y verificar nuestros programas**. En este punto, cabe destacar que algunas de las metodologías más utilizadas en la actualidad se basan en la utilización de pruebas para poder avanzar más rápidamente. En algunas metodologías extremas, es obligatorio implementar las pruebas con anterioridad para poder comenzar con la implementación.

La implementación de las pruebas puede ofrecer una visión objetiva e independiente del estado del software que hemos implementado. Nótese que, cuando los desarrolladores prueban el propio código que han generado, normalmente, no encuentran errores.

Cuando disponemos de un software sobre el cual hay que realizar un proceso de prueba, es necesario definir la estrategia de pruebas y el tipo de pruebas que queremos llevar a cabo sobre cada uno de los componentes que forman nuestro programa. Además, las pruebas pueden centrarse también en la arquitectura global de nuestro sistema y no únicamente en los módulos.



Fig.15. Pruebas de software.

9.1. Pruebas de caja blanca

Son unas técnicas de pruebas en las que **se analiza el diseño código y la estructura interna de cada componente**. Para ponerlas en práctica, los programadores deben conocer exactamente cómo ha sido creado el sistema para poder elaborar el conjunto de pruebas a llevar a cabo. Normalmente, se centran en probar determinados detalles del código fuente. Por ello, es necesario que los programadores conozcan detalles del código para poder realizar y ejecutar los casos de pruebas que sean necesarios. Uno de los objetivos fundamentales de las pruebas de caja blanca es poder probar el flujo de trabajo de las aplicaciones y asegurarse de que todo esté funcionando correctamente. Algunas de las técnicas más utilizadas para la implementación de pruebas de caja blanca son: La prueba de control de flujo, la de bifurcación, la de caminos básicos, etc.

9.2. Pruebas de caja negra

Las pruebas de caja negra o pruebas funcionales son un tipo de pruebas que se centran en **comprobar si los diferentes componentes del software funcionan correctamente sin conocer detalles de la implementación**. Reciben el nombre de pruebas de caja negra porque los programadores no conocen ningún detalle sobre la implementación del sistema, de hecho, para que sean eficaces, debe ser así. El programador únicamente puede conocer las funcionalidades que va a probar, pero no cómo están implementadas. El objetivo es minimizar los diferentes casos de pruebas que pueden ocurrir cuando implementamos pruebas de caja blanca. Para lograr una buena efectividad de las pruebas de caja negra es necesario poder elegir con certeza las condiciones de los valores de entrada y salida necesarios en cada una de las funcionalidades a probar. Existen diferentes técnicas para implementar las pruebas de caja negra, por ejemplo, podemos hablar de valores límites, equivalencias, etc.

9.3. Desarrollo guiado por pruebas (TDD)

El TDD (*Test Driven Development*) es una **estrategia de programación que se utiliza en diferentes metodologías software para obtener un software estable y verificado lo antes posible**. Normalmente, para aplicar esta estrategia, es necesario poder establecer pruebas unitarias con anterioridad que permitan analizar y probar cada una de las funcionalidades de nuestro sistema. Para poder implementar esta estrategia y realizar un proyecto bajo sus directivas, es necesario seguir diferentes **fases**:

- **Desarrollo y escritura de las pruebas**: Para este enfoque de desarrollo es necesario definir las pruebas del software antes de su implementación. La idea es que una vez que tengamos la funcionalidad implementada, seamos capaces de pasar satisfactoriamente las pruebas.

- **Validación de las pruebas**: Cuando una prueba falla, puede deberse a que haya un error en el software o bien en la propia prueba. Antes de comenzar el desarrollo, es necesario que validemos que las pruebas sean realmente válidas para la solución que estamos implementando.

- **Refactorización**: En esta fase vamos a simplificar el código limpiando aquellas partes que no sean necesarias y aunando en módulos comunes aquellas que sean redundantes.



Fig.16. Desarrollo guiado por pruebas.

/ 10. Resumen y resolución del caso práctico de la unidad

El desarrollo de un sistema libre de errores es prácticamente imposible. Como programadores disponemos de ciertas herramientas que podemos utilizar para mejorar la calidad de un producto disminuyendo los posibles errores que puedan existir. Cuando establecemos una **política de pruebas**, es necesario definir el tipo de pruebas que vamos a realizar, así como el objetivo que perseguimos con su realización.

En este tema hemos presentado el concepto de **depuración**, que hace posible observar con detalle el flujo de ejecución de un programa y manejar su ejecución a demanda del programador. Gracias al depurador somos capaces de resolver un gran número de problemas. Es importante tener en cuenta que el proceso de depuración puede consumir un tiempo significativo.

Por último, se ha conocido, además, el **depurador de código** que trae integrado **NetBeans**, donde se puede seguir la traza de una excepción para poder averiguar cuál es su causa, mediante los puntos de ruptura y la ventana de control de variables.

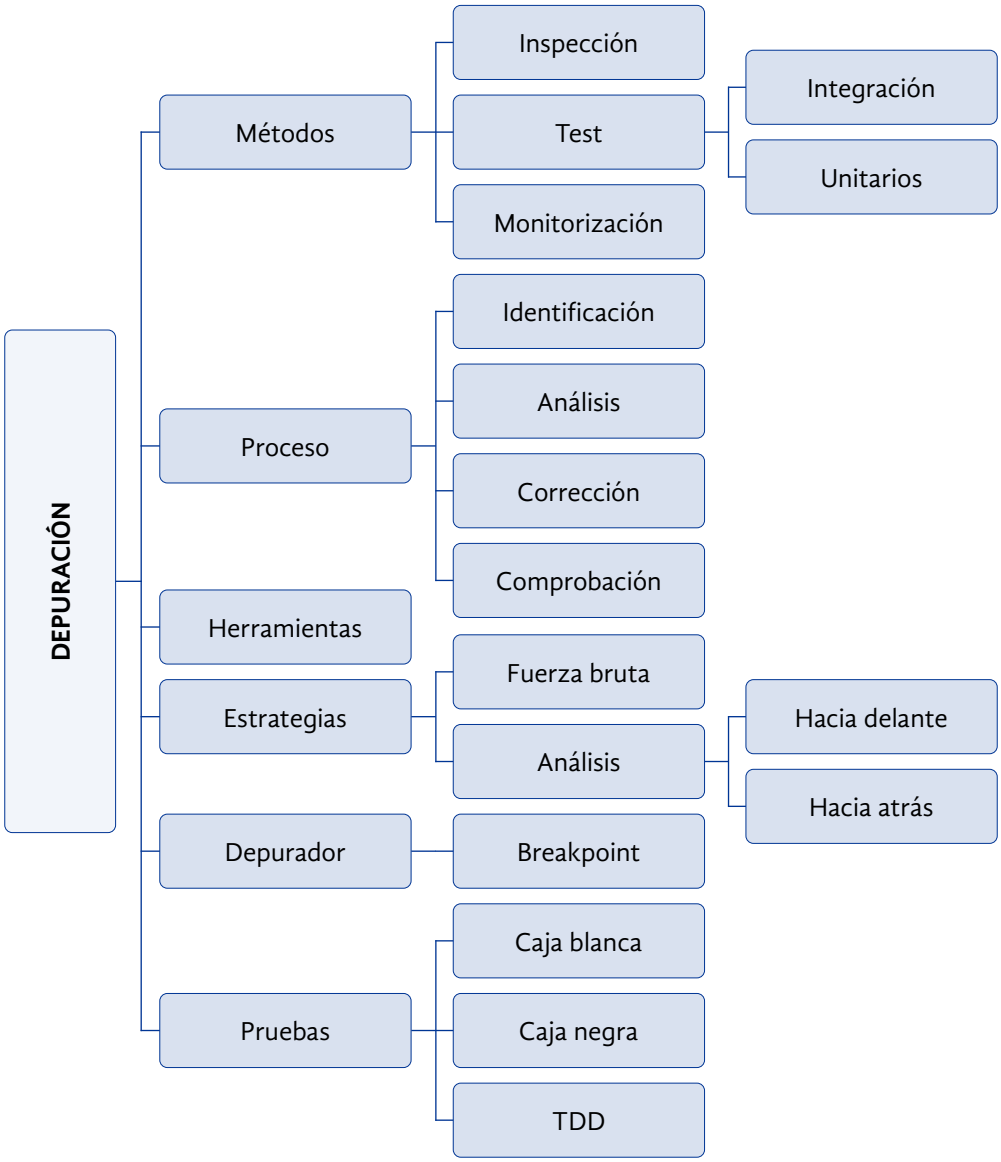


Fig.17. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Cuando se finaliza el desarrollo de un componente software, es necesario establecer unas pruebas mínimas que permitan garantizar que la funcionalidad que hemos implementado funciona correctamente. Una buena práctica es evitar que los desarrolladores lleven a cabo pruebas sobre componentes que han desarrollado ellos mismos. Normalmente, cuando lo hacen, no encuentran errores, aunque existan. Además, es muy complicado que el desarrollador pueda ignorar lo que ya conoce del código, lo que hace casi imposible efectuar pruebas de caja negra.

Precisamente, en el caso práctico, se nos pide que realicemos las pruebas sobre un componente que acabamos de desarrollar. Como hemos visto, esto no sería adecuado, porque vamos a ejecutar pruebas que no van a encontrar errores. Además, no sería posible realizar pruebas de caja negra, porque ya conocemos los detalles de implementación. Como mucho, podríamos efectuar pruebas de caja blanca.

/ 11. Bibliografía

Bhargava, A. Y. (2019). *Algoritmos: Una guía ilustrada para programadores y curiosos*. Madrid: Ediciones Anaya Multimedia (Grupo Anaya).

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Knuth, D. E. (1997). *The Art of Computer Programming* (Vol. 1: Fundamental Algorithms) (3.ª ed.). Addison-Wesley.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S.A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference* (11.ª ed.). McGraw-Hill Education.